File: `src/components/LineGraph.jsx`

Category: React Component (Chart Renderer / Visualization Engine)
Runtime: Browser (React + Chart.js)
Primary parent: `src/components/ChartPanel.jsx`
Key responsibilities: data windowing, downsampling, Y-axis range computation, Chart.js configuration

---

Overview

`LineGraph` is the low-level chart rendering component. It receives:

- a history of parsed data points (`dataPoints`)
- a list of selected series definitions (`series`)
- a time interval window (`intervalMs`, `nowMs`)
- optional Y-axis locking settings (`yAxis`)
- a callback for reporting auto Y-axis range (`onAutoYRange`)

It outputs a Chart.js line chart via `react-chartjs-2`.

Imports

React

- `useMemo`, `useEffect`, `useRef`

Chart.js / React wrapper

- Chart.js components:
  - `Chart as ChartJS`
  - `LinearScale`, `PointElement`, `LineElement`
  - `Title`, `Tooltip`, `Legend`
  - `TimeScale`
- `Line` from `react-chartjs-2`
- `"chartjs-adapter-date-fns"` (enables time scale formatting via date-fns)

Registration

```
ChartJS.register(LinearScale, PointElement, LineElement, Title, Tooltip,
Legend, TimeScale)
```

This is required because Chart.js v3+ uses explicit registration.

Internal Helper Function

`preserveExtremesEpochBuckets(points, bucketMs)`

Signature

```
preserveExtremesEpochBuckets(
  points: Array<{ timestamp: number, value: number }>,
  bucketMs: number
) → Array<{ timestamp: number, value: number }>
```

Purpose

Reduces the number of plotted points while preserving important shape features.

Instead of plotting every sample (which can be slow), it groups points into time buckets and keeps:

- first point in the bucket
- minimum value point in the bucket
- maximum value point in the bucket
- last point in the bucket

This maintains:

- spikes
- dips
- step changes
- general envelope of the signal

Algorithm Summary

1. Sort points by timestamp.
2. For each time bucket (`bucketMs` wide):
   - collect points in that bucket
   - compute first / last / min / max
   - de-duplicate identical points (timestamp + value)
3. Append selected candidates to output.

Side effects

None (pure function).

Exported Component

`LineGraph(props)`

Signature

```
LineGraph(props: LineGraphProps) → JSX.Element
```

Properties (`LineGraphProps`)

```
dataPoints: ParsedDataPoint[]
series: Array<{
  id: string
  key: string
  label: string
  unit?: string
  color: string
  tension?: number
  thickness?: number
  valueAccessor: (dp: ParsedDataPoint) → number | null
}>
timeUnit: "second" | "minute" | "hour" | "day"
intervalMs: number
chartWidthPx?: number (default 900)
desiredPxPerPoint?: number (default 10)
nowMs?: number
yAxis?: { locked: boolean, min: number | null, max: number | null }
onAutoYRange?: (min: number, max: number) → void
```

Internal Reference

`chartRef`

```
useRef(null)
```

A reference to the underlying chart instance created by `react-chartjs-2`.

Effect: Chart cleanup on unmount

Logic

- On unmount, attempts to call `destroy()` on the chart instance.

Signature

```
useEffect(() => cleanup, [])
```

Dependency array explanation

- Empty dependency array means this effect runs once:
  - setup on mount
  - cleanup on unmount

This helps prevent Chart.js instance leaks when components are removed.

Time window computation

`now` and `windowStart`

- `now` is either `nowMs` (if provided) or `Date.now()`
- `windowStart = now - intervalMs`

This defines the chart's visible time window.

Core computation (`useMemo`)

`computed = useMemo(() => {...}, [ ... ])`

This block builds everything needed for the chart:

- x-axis range (min/max)
- downsampled points for each series
- computed Y-axis range (min/max)
- Chart.js dataset objects
- combined y-axis title string

Dependency array used

```
[dataPoints, series, intervalMs, chartWidthPx, desiredPxPerPoint,
windowStart, now]
```

Why each dependency is included

- `dataPoints`: new samples require rebuilding datasets
- `series`: user selection/style changes require rebuilding datasets
- `intervalMs`: time window changes require rebuilding
- `chartWidthPx` / `desiredPxPerPoint`: affects downsampling resolution
- `windowStart` / `now`: shifts the visible window

Step 1: Horizontal padding

A small padding is added so endpoints are not visually clipped:

- Compute milliseconds per pixel
- Convert 1 pixel to milliseconds
- Expand xMin/xMax by that amount

Step 2: Empty-series fallback

If no series are selected:

- return empty datasets
- use default Y range
- avoid chart rendering

Step 3: Downsampling parameters

Compute maximum points based on chart width:

```
maxPoints = max(10, floor(chartWidthPx / desiredPxPerPoint))
bucketMs = max(floor(intervalMs / maxPoints), 1)
```

This ensures:

- long time windows do not render thousands of points
- bucket size never becomes 0

Step 4: Build per-series points

For each series definition:

1. Convert each `ParsedDataPoint` into `{timestamp, value}`
2. Filter:
   - outside time window
   - null values
   - NaN values
3. Apply bucket downsampling: `preserveExtremesEpochBuckets`

Step 5: Compute auto Y-axis range

1. Gather all values from all series
2. Find rawMin and rawMax
3. Compute range padding:
   - `range = rawMax - rawMin` (fallback 1)
   - `computedYMin = max(rawMin - range * 0.5, 0)`
   - `computedYMax = rawMax + range * 0.5`

Note: minimum is clamped to 0. This assumes metrics are non-negative. (Valid for voltage/current/power/energy; less ideal for metrics that could go negative.)

Step 6: Build Chart.js datasets

Each dataset contains:

- label including unit
- x/y points
- stroke color and thickness
- interpolation tension
- no point markers (`pointRadius: 0`)

Step 7: y-axis title

Builds a combined label string, e.g.:

- `"Voltage (R) V, Current (∑) A"`

This is used as the Y-axis label.

Y-axis lock handling

After computing auto range, the component selects the effective Y range:

- If locked AND provided min/max are finite → use locked values
- Else → use computedYMin/computedYMax

Effect: Report auto Y-range to parent

```
useEffect(() => { onAutoYRange(computedYMin, computedYMax) }, [computedYMin,
computedYMax, onAutoYRange])
```

Purpose
Allows the chart panel state manager to store the latest auto range, enabling the lock feature.

Dependency array explanation

- Runs when the computed min/max change
- Runs when a new callback reference is provided

Chart.js options (`useMemo`)

Options include:

- responsiveness
- disabled animations
- time axis formatting (seconds for short intervals)
- x-axis bounds set to computed min/max
- y-axis bounds set to effective min/max
- y-axis title set to combined label string

Dependency array includes values that influence axis formatting and bounds.

Conditional rendering: no series selected

If `series.length === 0`, the component renders a placeholder message instead of the chart.

Side effects

- Creates and destroys Chart.js instances (managed by React wrapper)
- Calls `onAutoYRange` callback when auto range changes

No network activity, no timers.